



Program Studi Teknik Elektro ITB

Nama Kuliah (Kode) : EL2008-Pemecahan Masalah dengan Pemrograman

Tahun / Semester : 2024-2025 / Genap

Modul : 01 – Overview of C Language

Nama Asisten / NIM :

Nama Praktikan / NIM : Muhammad Azka Rahadian / 13224036

Tugas Pendahuluan

Soal Konsep

Library

- Stdlib.h → berisi fungsi umum dalam bahasa c terutama untuk manajemen memori, kontrol proses, dan utilitas lainnya. Sebagai contoh :
 - `int *arr = (int*) malloc(5 * sizeof(int));`
 - `arr = (int*) realloc(arr, 10 * sizeof(int));`
- Stdio.h → library ini berfungsi untuk operasi input dan output. Sebagai contoh :
 - `scanf("%d", &x);`
 - `printf("Hello World");`
- Math.h → library ini berfungsi untuk menyediakan fungsi matematika yang dapat digunakan seperti function. Sebagai contoh :
 - Akar → `double x = sqrt(16);`
 - Kuadrat → `double y = pow(2,3);`
- String.h → Library ini digunakan untuk melakukan manipulasi string. Sebagai contoh :
 - `Strcpy (destination, source);`
 - `Int len = strlen("Praktikum PMP");`

Operator

- Arithmetic Operators → Operator ini digunakan untuk melakukan matematika

Tabel 1 Arithmetic Operators

Operator	Deskripsi	Contoh
+	Melakukan Penjumlahan	$A + B = 15$
-	Melakukan Pengurangan	$A - B = 5$
*	Melakukan Perkalian	$A * B = 20$

- Relational Operators → Simbol yang menandakan hubungan dari 2 buah nilai yang dibandingkan

Tabel 2 Relational Operators

Operator	Deskripsi	Contoh
==	Membandingkan kedua nilai apakah sama. Jika sama maka dihasilkan True	$(A == B)$ is False
!=	Membandingkan kedua nilai apakah berbeda. Jika kedua nilai berbeda maka hasilnya True	$(A != B)$ is True
>	Membandingkan apakah nilai di sebelah kiri lebih besar dari kanan. Jika benar maka True	$(A > B)$ is False

- c. Logical Operators → Simbol yang menandakan perintah untuk melakukan operasi kondisi logika.

Tabel 3 Logical Operators

Operator	Deskripsi	Contoh
&&	Operator logika AND. Ketika kedua kondisi bernilai True atau tidak 0 maka akan dihasilkan kondisi True	(A&&B) is False
	Operator logika OR. Ketika salah satu dari kedua kondisi bernilai true atau tidak 0, maka hasilnya True	(A B) is True

- d. Bitwise Operators → Simbol untuk melakukan perintah operasi pada domain biner

Tabel 4 Bitwise Operators

Operator	Deskripsi
&	Operator biner AND
	Operator biner OR
^	Operator biner XOR
~	Operator biner NOT atau negasi

- e. Assignment Operators → simbol perintah untuk menyimpan dan melakukan operasi

Tabel 5 Assignment Operators

Operator	Deskripsi	Contoh
=	Operator standar assignment, menyimpan nilai dari operasi di sisi kanan ke variabel di kiri	C= A+B
+=	Melakukan operasi penjumlahan antara sisi kanan dan kiri kemudian menyimpan nilainya pada variabel sisi kiri	C += A sama dengan C = C + A
-=	Melakukan pengurangan antara sisi kanan dan kiri kemudian menyimpan nilainya pada sisi kiri	C -= A sama dengan C = C-A

- f. Other Operators

Tabel 6 Other Operators

Operator	Deskripsi	Contoh
&	Me-return alamat memori dari variabel yang digunakan	Scanf("%d", &i);
*	Pointer ke alamat memori suatu variabel	*a

- g. Contoh penggunaan operator beserta input dan output

```

1 #include <stdio.h>
2 int main() {
3     int a = 6, b = 4;
4     //arithmetic
5     printf("a + b = %d\n", a+b); //10
6
7     //relational
8     printf("a > b = %d\n", a>b); //1
9
10    //logical
11    printf("(a > 3 && b < 5) = %d\n", (a>3 && b<5)); //1
12
13    //assignment

```

```

14     a+=2;
15     printf("a setelah += 2 = %d\n", a); //8
16
17     //bitwise
18     printf("a & b = %d\n", a&b); //operasi bit
19     return 0;
20 }

```

Looping

a. Jenis dan perbedaan Loop

Tabel 7 Jenis Loop

Jenis	Deskripsi
For Loop	Digunakan ketika jumlah perulangan sudah diketahui sejak awal. Memiliki tiga komponen utama yaitu inisialisasi, kondisi, dan increment/decrement.
While Loop	Digunakan ketika jumlah perulangan tidak diketahui secara pasti dan bergantung pada kondisi tertentu. Kondisi diperiksa di awal, sehingga perulangan bisa saja tidak dijalankan sama sekali jika kondisi awal bernilai salah.
Do-While Loop	Mirip dengan while loop, tetapi kondisi diperiksa di akhir. Hal ini menyebabkan blok kode di dalamnya pasti dijalankan minimal satu kali, meskipun kondisi awal bernilai salah.

b. Situasi Loop

While loop sebaiknya digunakan ketika jumlah iterasi tidak diketahui secara pasti sejak awal dan bergantung pada kondisi yang dinamis, seperti input dari pengguna atau proses yang terus berjalan hingga kondisi tertentu terpenuhi. Sementara for loop yang biasanya digunakan ketika jumlah perulangan sudah jelas, while loop lebih fleksibel karena hanya membutuhkan kondisi sebagai acuan berhenti.

Tipe Data

a. Jenis dan perbedaan Tipe Data

Tabel 7 Jenis Tipe Data

Jenis Tipe Data	Deskripsi
Integer	Digunakan untuk menyimpan bilangan bulat tanpa bagian desimal. Contoh: int, short, long.
Floating-point	Digunakan untuk menyimpan bilangan yang memiliki bagian desimal. Contoh: float, double.
Character	Digunakan untuk menyimpan satu karakter. Contoh: char.
Derived data type	Tipe data turunan yang dibentuk dari tipe dasar, seperti array, pointer, dan string (array of char).

b. Situasi Tipe Data

- o Integer (int) digunakan ketika data yang disimpan berupa bilangan bulat, seperti jumlah mahasiswa, jumlah barang, atau indeks perulangan.

- Floating-point (float atau double) digunakan ketika data memiliki nilai pecahan atau membutuhkan presisi, seperti nilai suhu, tegangan listrik, atau hasil perhitungan matematika.
- Character (char) digunakan untuk menyimpan satu karakter, seperti huruf, simbol, atau input pilihan dari pengguna.
- Array digunakan untuk menyimpan banyak data dengan tipe yang sama dalam satu variabel, misalnya daftar nilai mahasiswa.
- Pointer digunakan untuk menyimpan alamat memori dari variabel lain, yang berguna dalam pengelolaan memori dan passing parameter secara reference.

Fungsi dan Rekursi

- a. Apa yang akan terjadi apabila suatu implementasi rekursi tidak memiliki base case? Jika suatu fungsi rekursif tidak memiliki base case, maka fungsi tersebut akan terus memanggil dirinya sendiri tanpa henti. Hal ini akan menyebabkan:
 1. Infinite recursion
Fungsi akan terus berjalan tanpa pernah berhenti.
 2. Stack overflow
Setiap pemanggilan fungsi akan menggunakan stack memory. Tanpa base case, stack akan terus terisi hingga penuh.
 3. Program crash atau error
Ketika stack sudah penuh, program akan berhenti secara paksa (runtime error).
- b. Kelebihan dan kekurangan rekursi dibandingkan iterasi dalam pemrograman

Kelebihan	Kekurangan
Kode lebih sederhana dan ringkas, sehingga lebih mudah dipahami terutama untuk masalah yang memiliki pola berulang	Penggunaan memori lebih besar karena setiap pemanggilan fungsi disimpan dalam stack
Cocok untuk menyelesaikan masalah tertentu seperti tree, divide and conquer, dan faktorial	Performa cenderung lebih lambat dibandingkan iterasi karena adanya overhead pemanggilan fungsi
Struktur program lebih modular dan sesuai dengan definisi matematis dari suatu masalah	Rentan terjadi infinite recursion jika tidak terdapat base case yang tepat

Soal Pemrograman

Bagian 1: Basic Sorting

```

1  /*
2  TP 1 - Bagian 1
3  Nama : Muhammad Azka Rahadian
4  NIM : 13224036
5  */
6  #include <stdio.h>
7  int main () {
8      int n;
9      printf ("Masukkan jumlah bilangan: ");
10     scanf ("%d", &n);

```

```

11     int arr[n];
12     for (int i=0; i<n; i++){
13         printf("Masukkan bilangan ke-%d: ", i+1);
14         scanf("%d", &arr[i]);
15     }
16     //bubble sort -> swap kanan
17     int temp;
18     for (int i=0; i<n-1; i++){
19         for(int j=0; j<n-i-1; j++){
20             if (arr[j]>arr[j+1]){
21                 //swap
22                 temp = arr[j];
23                 arr[j]= arr[j+1];
24                 arr[j+1] = temp;
25             }
26         }
27     }
28     //print
29     printf("Bilangan setelah diurutkan: ");
30     for (int i=0; i<n; i++){
31         printf("%d ", arr[i]);
32     }
33     return 0;
34 }

```

Bagian 2: Sequence Angka dengan Rekursif

```

1  /*
2  TP 1 - Bagian 2
3  Nama : Muhammad Azka Rahadian
4  NIM : 13224036
5  */
6  #include <stdio.h>
7  #include <string.h>
8
9  int visited[1000]={0};
10 int sumSquare(int n){
11     int sum =0;
12     int digit;
13     while(n>0){
14         digit = n%10;
15         sum += digit*digit;
16         n=n/10;
17     }
18     return sum;
19 }
20 void proses(int n, int count){
21     if (n==1){
22         printf("%d", count);
23         return;
24     }
25     if (visited[n]==1){
26         printf("infinity");
27         return;
28     }
29     visited[n]=1;
30     int next = sumSquare(n);
31     proses(next, count+1);
32 }
33 int main() {

```

```
34     int n;  
35     printf("Masukkan N: ");  
36     scanf("%d", &n);  
37     proses(n,0);  
38     return 0;  
39 }
```